Un puntero guarda una dirección

& obtiene la dirección; * permite acceder al objeto apuntado.

Ejemplo visual

`edad` guarda `20`; `ptr` guarda la dirección de `edad`. Las direcciones de esta lámina son ilustrativas.

```
int edad = 20;
int* ptr = &edad;
```



Expresiones relacionadas

Expresión	Qué pide	En este ejemplo
`edad`	El valor de edad	`20`
`&edad`	La dirección de edad	`0x1000`
`ptr`	La dirección guardada	`0x1000`
`*ptr`	El valor del objeto apuntado	`20`

Después de inicializar `ptr`: `ptr == &edad` y `*ptr == edad`.

Dos sentidos de *

Al declarar un puntero

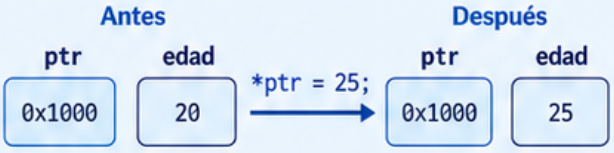
```
int* ptr = &edad;
```

Al declarar, `*` indica que `ptr` es un puntero a `int`.

Al acceder a través del puntero

```
*ptr = 25;
```

En una expresión, `*ptr` designa a `edad`: permite leerla o modificarla.



Cambia `edad`; la dirección guardada en `ptr` se conserva.

Ejemplo completo

```
#include <iostream>
int main() {
  int edad = 20;
  int* ptr = &edad;
  std::cout << edad << '\n';
  std::cout << &edad << '\n';
  std::cout << ptr << '\n';
  std::cout << *ptr << '\n';
  *ptr = 25;
  std::cout << edad << '\n';
}
```

Salida (explicación)

- `20` · el valor de `edad`
- dirección de edad · la dirección de `edad`
- la misma dirección · la mirección guardada en `ptr`
- `20` · valor a través de `*ptr`
- `25` · valor después de modificar con `*ptr`

Analogía

Como una casa: `edad` es el contenido; `&edad` es la dirección; `ptr` es la nota con esa dirección; `*ptr` permite acceder al contenido.

Recuerda

Un puntero debe apuntar a un objeto válido para leer o escribir mediante `*`. No desreferencies `nullptr` ni un puntero sin inicializar.

C++20: [expr.unary.op], [dcl.ptr].

Punteros y cadenas de caracteres

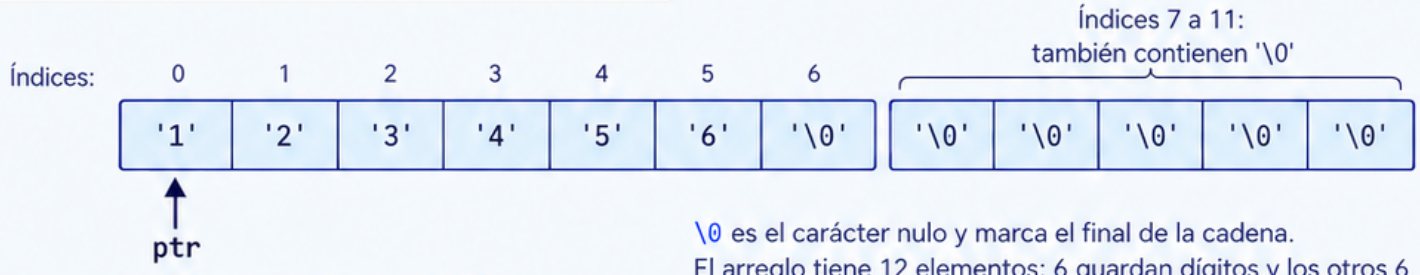
char* apunta a un carácter de la cadena

El arreglo guarda caracteres; el puntero permite acceder a ellos.

Arreglo y puntero a la cadena

```
char matricula[12] = "123456";
char* ptr = matricula;
```

En esta inicialización, `matricula` se convierte en un puntero a su primer elemento: `ptr == &matricula[0]`.
El arreglo y el puntero son objetos distintos.



`\0` es el carácter nulo y marca el final de la cadena.
El arreglo tiene 12 elementos; 6 guardan dígitos y los otros 6 se inicializan en cero.

Leer un char*

`cout` trata `char*` como una cadena y lee hasta `\0`.
`*ptr` es solo el primer carácter. La conversión a `const void*` permite mostrar la dirección.

```
std::cout << ptr;    // 123456
std::cout << *ptr;   // 1
std::cout << static_cast<const void*>(ptr);
```

Getter sin copia del arreglo

Se devvelve un puntero al primer carácter; no se copia el arreglo.

```
class Estudiante {
   char matricula[12] = "123456";
public:
    const char* getMatricula() const;
};

const char* Estudiante::getMatricula() const {
    return matricula;
}
```

Diagrama del objeto y el puntero



Uso del getter

```
Estudiante e;
const char* p = e.getMatricula();

std::cout << p;    // 123456
std::cout << *p;   // 1
```

Vida útil y seguridad

El puntero devuelto puede usarse mientras `e` exista y su almacenamiento siga siendo válido. Al destruir `e`, `p` queda colgante. Antes de imprimir una cadena, debe existir un `\0` accesible dentro del almacenamiento válido.

Dos calificadores const

`const char*`
impide modificar los caracteres mediante el puntero devuelto.

`const` después de `()`
indica un método que no modifica el estado del objeto.

C++20: [conv.array], [dcl.init.string], [ostream.inserters.character], [basic.life].